

Sesion1 - Introduccion

Fundamentos de los Lenguajes de Programacion

Que es un lenguaje de Programacion?

R = Un lenguaje de programación es un conjunto de instrucciones que permiten a los programadores comunicar de forma efectiva con las máquinas para ejecutar tareas específicas.

Cual es el objetivo principal de los lenguajes de programacion?

R = El objetivo principal de los lenguajes de programación es facilitar la creación de programas mediante instrucciones que un sistema puede procesar, permitiendo el control del hardware y la realización de operaciones complejas.

Que son los lenguajes de bajo nivel?

R = Más cercanos al lenguaje de la máquina. Incluyen ensambladores y lenguajes como C, que permiten un control directo sobre el hardware.

Que son los lenguajes de alto nivel?

R = Más abstractos y fáciles de usar para los humanos. Ejemplos incluyen Python, Java y JavaScript, que ofrecen construcciones más intuitivas y son menos propensos a errores.

Cuales son los objetivos de los lenguajes de Programacion?

R =

- Facilitar la comunicacion con la computadora: Proporcionan una sintaxis y semántica que permite a los desarrolladores expresar soluciones a problemas complejos de manera comprensible.
- Abstraccion: Permiten a los programadores trabajar en un nivel más alto, enfocándose en la lógica y el diseño del software en lugar de en detalles del hardware.
- Portabilidad: Muchos lenguajes modernos están diseñados para ser portables, lo que significa que el mismo código puede ejecutarse en diferentes plataformas con mínimas modificaciones.
- Mantenimiento y legibilidad: Lenguajes como Python y Java están diseñados para ser legibles y fáciles de mantener, lo que facilita el trabajo colaborativo y la evolución del software.
- Eficiencia: Los lenguajes de programación permiten optimizar el rendimiento de las aplicaciones, tanto en términos de velocidad como de uso de recursos.
- Ecosistema y comunidad: Los lenguajes de programación suelen estar acompañados de bibliotecas, frameworks y una comunidad activa que proporciona soporte y recursos, lo que

acelera el desarrollo y la innovación.

Compilacion e Interpretacion de Lenguajes

Que es la compilacion?

R = La compilación es el proceso mediante el cual un compilador traduce el código fuente completo de un programa escrito en un lenguaje de alto nivel a un lenguaje de bajo nivel o código máquina, que es directamente ejecutable por el sistema operativo de la computadora.

Cuales son las características de la compilacion?

R =

- Ejecución: Una vez que el código ha sido compilado, el programa resultante puede ser ejecutado varias veces sin necesidad de recompilarlo, lo que generalmente hace que la ejecución sea más rápida.
- Errores: Todos los errores de sintaxis y semántica son detectados durante la fase de compilación, lo que permite a los programadores corregirlos antes de ejecutar el programa.

Que es la interpretacion?

R = Es el proceso en el cual un intérprete traduce el código fuente línea por línea durante la ejecución del programa.

Cuales son las características de la interpretacion?

R =

- Ejecución: Dado que el código se traduce y ejecuta simultáneamente, los programas interpretados suelen ser más lentos en comparación con los compilados.
- Errores: Los errores de ejecución pueden surgir durante la interpretación, lo que significa que un programa puede fallar en medio de su ejecución, haciendo que la detección de errores sea menos eficiente.

Historia y Evolucion de los Lenguajes de Programacion

- 1940-1950: En esta época, los programadores utilizaban código de máquina para comunicarse directamente con el hardware. Esto requería un profundo conocimiento de la arquitectura de la computadora, lo que hacía que la programación fuera complicada y propensa a errores.
- FORTRAN (1957): Considerado el primer lenguaje de programación de alto nivel, FORTRAN fue diseñado para cálculos científicos y matemáticos. Su éxito radicó en su capacidad para

simplificar el proceso de programación, permitiendo a los científicos centrarse en sus problemas en lugar de en la sintaxis del lenguaje.

- COBOL (1959): Orientado a la gestión de datos empresariales, COBOL se convirtió en el estándar para aplicaciones comerciales. Fue uno de los primeros lenguajes en permitir la manipulación de registros y archivos, lo que facilitó el desarrollo de software para negocios.
- C (1972): Desarrollado por Dennis Ritchie, C combinó la eficiencia del lenguaje de máquina con una sintaxis más legible. Este lenguaje se convirtió en la base para muchos sistemas operativos, incluido UNIX, y sentó las bases para lenguajes posteriores como C++ y C#.
- Pascal (1970): Diseñado como herramienta educativa, Pascal promovía buenas prácticas de programación estructurada. Su enfoque en la claridad y la organización influyó en la enseñanza de la programación en todo el mundo.

Paradigmas de Programacion

Que son los paradigmas de Programacion?

R = Son enfoques teóricos sobre cómo resolver problemas mediante el código.

Sesion2 - Paradigmas de Lenguajes de Programacion

Cuales son los paradigmas de lenguajes de programacion?

R =

- Programacion Imperativa
- Programacion Orientada a Objetos
- Programacion Funcional
- Programacion Logica
- Programacion Declarativa
- Programacion Concurrente

En que consiste la programacion Imperativa?

R = se basa en la idea de dar instrucciones explícitas a la computadora sobre cómo realizar una tarea.

Que es la programacion orientada a objetos?

R = La programación orientada a objetos es un paradigma que organiza el código en objeto", que son instancias de clases.

Que es la programacion funcional?

R = se basa en la idea de tratar la computación como la evaluación de funciones matemáticas.

Que es la programacion logica?

R = es un paradigma que se basa en la formalización de problemas a través de relaciones lógicas.

Que es la programacion Declarativa?

R = es un enfoque que se centra en el "qué" se quiere lograr en lugar del "cómo" hacerlo.

Que es la programacion Concurrente?

R = se refiere al paradigma que permite la ejecución simultánea de múltiples procesos o hilos.

Cuales son los criterios para la seleccion de un lenguaje?

R =

- El tipo de aplicacion que se quiere desarrollar con el lenguaje
- Rendimiento necesario
- Comunidad y Ecosistema
- Portabilidad
- Facilidad de mantenimiento

Sesión 03 — Compilación e Interpretación (I)

1.  ¿Qué hace un compilador “puro”?
 Traduce el programa fuente completo a un **programa objetivo** (normalmente código máquina) y desaparece; luego el SO ejecuta ese programa objetivo. El **lugar de control** durante la ejecución pasa a ser el programa objetivo.
2.  ¿Qué hace un intérprete durante la ejecución?
 Permanece presente y **controla** la ejecución, leyendo sentencias (casi) una por una en el lenguaje de alto nivel y ejecutándolas sobre la marcha, actuando como una **máquina virtual**.
3.  ¿Por qué suele ofrecer mejor diagnóstico la interpretación?
 Porque ejecuta el **código fuente directamente**, por lo que puede brindar depuración a nivel de fuente y mensajes de error más cercanos al programa original.
4.  ¿Qué tipo de características favorecen la interpretación?
 Aquellas con **decisiones en tiempo de ejecución** (tamaños y tipos dependientes de datos, nombres dinámicos) y **auto-modificación** (programa que genera/ejecuta código nuevo), como en Lisp o Prolog.
5.  ¿Qué es el **enlace tardío** (late binding)?
 Retrasar decisiones de implementación hasta **tiempo de ejecución**; es típico en intérpretes y lenguajes dinámicos.

6. ? ¿Cuál es la ventaja clásica de la compilación?
 - 💬 **Rendimiento**: decide cosas en compilación (p. ej., ubicación fija de una variable) y evita búsquedas/chequeos repetidos en ejecución.
 7. ? ¿Por qué la distinción “compilado vs interpretado” puede ser confusa?
 - 💬 Muchas implementaciones **mezclan** ambos (traductor inicial + VM). En Java, por ejemplo, un compilador produce **bytecode** que ejecuta una **máquina virtual**.
 8. ? ¿Cuándo decimos que un lenguaje “está compilado”?
 - 💬 Cuando el traductor **analiza a fondo** y hace transformaciones **no triviales**, produciendo una forma intermedia poco parecida al código fuente.
 9. ? ¿Qué es el **preprocesamiento** y qué hace?
 - 💬 Un traductor inicial que **elimina comentarios/espacios**, agrupa caracteres en **tokens**, puede expandir **macros** y reconocer estructuras de alto nivel para producir una forma intermedia más eficiente de interpretar.
 10. ? ¿Qué son las **rutinas de biblioteca** y el **enlazador**?
 - 💬 Funciones externas (I/O, trigonometría, etc.). El **linker** combina el código del programa con esas rutinas para producir el ejecutable final.
 11. ? ¿Dónde aparece “interpretación” dentro de bibliotecas compiladas?
 - 💬 En formateo de salida (Fortran) o `printf` de C, que **interpretan** cadenas de formato en tiempo de ejecución.
 12. ? ¿Resumen para el examen?
 - 💬 Compilación = rendimiento y decisiones tempranas; interpretación = flexibilidad y diagnóstico; en la práctica, **híbridos** con preprocesamiento, bibliotecas y VM.
-

Sesión 04 — Compilación e Interpretación (II)

1. ? ¿Por qué muchos compiladores generan **ensamblador** y no directamente máquina?
 - 💬 Facilita la **depuración** y aísla al compilador de cambios en el formato binario del SO; solo se actualiza el ensamblador común.
2. ? ¿Qué hace el **preprocesador de C**?
 - 💬 Elimina comentarios, **expande macros** y permite **compilación condicional** para generar variantes desde la misma fuente.
3. ? ¿Qué es “traducción fuente-a-fuente” (ej. C++→C)?
 - 💬 Un **compilador real** de C++ hacía análisis completo y generaba C intermedio que luego compilaba un compilador de C, **oculto** para el usuario.
4. ? ¿Por qué no llamar “preprocesador” a ese compilador C++?
 - 💬 Porque un preprocesador no **comprende** el código; solo hace sustituciones. El compilador C++ sí hacía análisis sintáctico y semántico.

5. ? ¿Qué es **bootstrapping** de un compilador?
 - ☞ Proceso para **arrancar** un compilador en una plataforma usando un intérprete/forma intermedia (ej. **código P** para Pascal) y luego compilarse a sí mismo.
 6. ? ¿Por qué conservar intérprete y forma intermedia tras bootstrapping?
 - ☞ Ocupan **menos memoria** y ofrecen **mejores diagnósticos** y **ciclos rápidos** (ejecutar sin recompilar), útiles en desarrollo.
 7. ? ¿Cómo se “compilan” lenguajes tradicionalmente interpretados?
 - ☞ El compilador puede generar código que **asume** decisiones dinámicas; si la suposición falla, se **verifica** en runtime y se vuelve al intérprete.
 8. ? ¿Qué es **JIT** (compilación justo a tiempo) en Java/C#?
 - ☞ Compilar **bytecode/CIL** a máquina **inmediatamente antes** de ejecutar, combinando portabilidad y rendimiento.
 9. ? ¿Qué es el **microcódigo (firmware)**?
 - ☞ Un **intérprete** de bajo nivel grabado en ROM que ejecuta el set de instrucciones de alto nivel (especialmente en hardware previo a 1985).
 10. ? ¿Ejemplos no obvios de compiladores/Intérpretes?
 - ☞ TEX/troff (compilan descripciones de documento a **comandos de impresora**) y PostScript (intérprete en impresoras).
-

Sesión 05 — Una visión general de la compilación

1. ? ¿Cómo se organiza un compilador típico?
 - ☞ En **fases**: cada una descubre info o transforma el programa para la siguiente; front-end (hasta semántica) y back-end (generación/optimización).
2. ? ¿Qué es un “**pase**”?
 - ☞ Conjunto de fases serializadas: inicia tras terminar las previas y concluye antes de las siguientes; a veces es un programa independiente.
3. ? ¿Por qué dividir en pases?
 - ☞ Reutilizar **front-ends** para distintos targets o **back-ends** para distintos lenguajes; y por **memoria** en máquinas antiguas.
4. ? ¿Qué hace el **análisis léxico**?
 - ☞ Lee caracteres y los agrupa en **tokens** (palabras clave, identificadores, números, símbolos), reduciendo entrada y quitando comentarios.
5. ? ¿Qué hace el **análisis sintáctico**?
 - ☞ Organiza tokens en un **árbol de análisis** según una **gramática libre de contexto** (CFG).
6. ? ¿Qué valida el parser?
 - ☞ Que tokens estén **bien formados** y su secuencia respete la **sintaxis** de la gramática; de

lo contrario, reporta errores.

7. **?** ¿Qué es el **análisis semántico**?
 - ☞ Descubre el **significado**: relaciona identificadores a entidades, verifica **consistencia de tipos** y reglas que no están en la CFG, usando **tabla de símbolos**.
 8. **?** ¿Qué reglas se verifican en semántica?
 - ☞ Declaración previa, llamada con parámetros correctos, tipos compatibles, etiquetas únicas en `case`, etc.
 9. **?** ¿Qué es **semántica estática** vs **dinámica**?
 - ☞ Estática: verificable en compilación; Dinámica: requiere **chequeos en ejecución** (subíndices válidos, punteros válidos, no overflow).
 10. **?** ¿Qué es el **AST** y para qué sirve?
 - ☞ Árbol de **sintaxis abstracta**: simplifica el árbol concreto y se **anota** con tipos/entradas de la tabla; suele ser la **forma intermedia**.
 11. **?** ¿Cómo es la **generación de código**?
 - ☞ Traduce la forma intermedia al **lenguaje de destino**, asigna ubicaciones a variables y emite cargas/almacenamientos/ramas.
 12. **?** ¿Qué es la **optimización** (mejora de código)?
 - ☞ Transformaciones para ejecutar **más rápido** o con **menos memoria**; unas son independientes de máquina y otras específicas del target.
-

Sesión 06 — Sintaxis del lenguaje de programación

1. **?** ¿Por qué separar **sintaxis** y **semántica**?
 - ☞ Porque lenguajes con **semánticas similares** pueden tener **sintaxis distintas**; además, existen algoritmos **muy eficientes** para descubrir la **estructura sintáctica** y guiar el resto del proceso.
2. **?** ¿Qué ejemplifica la notación de números arábigos?
 - ☞ Uso de **dígitos** y del **cierre de Kleene (*)** para indicar repeticiones (cero o más) en la definición formal de cadenas válidas (p. ej. números).
3. **?** ¿Qué tres operaciones generan **lenguajes regulares**?
 - ☞ **Concatenación**, **alternancia (|)** y **cierre de Kleene (*)**. Con ellas se definen tokens y los reconocen los **escáneres**.
4. **?** ¿Cuándo necesitamos **recursión**?
 - ☞ Para definir estructuras **anidadas** (p. ej. paréntesis balanceados, expresiones), lo que lleva a **gramáticas libres de contexto** (CFL/CFG) reconocidas por **parsers**.
5. **?** ¿Qué son **tokens** típicos?
 - ☞ Palabras clave, identificadores, literales (enteros, reales, cadenas), símbolos y

comentarios; el formato suele ser **libre** (espacios suelen ignorarse).

6. ? Define **expresión regular** (formas básicas).
 - 💬 Un carácter; cadena vacía; concatenación; alternancia con `|` ; y cierre con `*` . Se usan **paréntesis** para desambiguar.
 7. ? ¿Por qué regex no “se definen a sí mismas”?
 - 💬 Las definiciones **no son recursivas**; para autorreferencia necesitamos **CFG**.
 8. ? ¿Qué introduce la **BNF/EBNF**?
 - 💬 BNF define CFG; EBNF añade comodidades como **Kleene plus (+)** y paréntesis “de meta-nivel”. El **símbolo de inicio** nombra la construcción general.
 9. ? ¿Qué tratan distinto parser y semántica respecto a tokens?
 - 💬 El parser **no distingue** grafías específicas de un identificador; el análisis **semántico sí**, por lo que el escáner **conserva la ortografía**.
-

Sesión 07 — Derivaciones y árboles de análisis sintáctico

1. ? ¿Cómo genera cadenas válidas una **CFG**?
 - 💬 Desde el **símbolo de inicio**, eligiendo producciones y **reemplazando no terminales** hasta obtener solo **terminales**.
2. ? ¿Qué es una **derivación** y una **forma sentencial**?
 - 💬 Derivación: secuencia de reemplazos que llega a terminales. Cada cadena intermedia es una **forma sentencial**; el resultado final es el **yield**.
3. ? ¿Qué denota $\Rightarrow^*$?
 - 💬 Que una cadena es el **resultado** después de **cero o más** reemplazos (pasos intermedios omitidos).
4. ? ¿Qué es derivación “más a la derecha” vs “más a la izquierda”?
 - 💬 En cada paso se expande el **no terminal más a la derecha** (o el más a la izquierda). Muchos parsers buscan una de estas dos.
5. ? ¿Cómo se estructura un **árbol de análisis**?
 - 💬 Raíz = **símbolo de inicio**; hojas = **yield**; cada nodo interno con sus hijos representa el **uso de una producción**.
6. ? ¿Qué implica que existan **dos árboles distintos** para la misma cadena?
 - 💬 **Ambigüedad** de la gramática; es un problema práctico para construir analizadores (se requieren reglas extra o desambiguación).